

Punto 2.9 - Índices y restricciones

Ejercicio guiado: diseñar índices simples, compuestos, filtrados y con columnas incluidas para los patrones de consulta del proyecto AceriaData, y añadir restricciones CHECK que protejan reglas de integridad directamente en SQL Server.

Contexto del proyecto: 2.9 parte íntegramente de 2.8. Las claves primarias y alternativas ya están resueltas y no se duplican con índices equivalentes. En este punto se añaden estructuras de acceso y restricciones de integridad: índices sobre OrdenFabricacion, PlanchaAcero, Aleacion, CertificadoCalidad y OrdenAleacion, además de CHECK para dimensiones/peso de planchas y porcentajes de aleaciones. Los defaults existentes proceden de 2.7 y las alternate keys de 2.8.

Objetivos de aprendizaje

- Diferenciar una clave candidata de un índice creado para acelerar consultas.
- Crear índices simples con `HasIndex`.
- Crear índices compuestos y razonar sobre el orden de sus columnas.
- Crear índices filtrados con `HasFilter`.
- Crear índices con columnas incluidas mediante `IncludeProperties`.
- Evitar índices redundantes cuando otro índice comienza por la misma columna.
- Comprender por qué la PK compuesta no sustituye un índice que empieza por `AleacionId`.
- Configurar CHECK constraints mediante `HasCheckConstraint`.
- Interpretar la migración real `M2_2_9`.
- Verificar índices, filtros, columnas incluidas y CHECK en SQL Server LocalDB.
- Probar que una restricción CHECK rechaza datos inválidos.
- Entender que un índice puede ayudar a una consulta sin garantizar que SQL Server vaya a elegirlo.

Paso 1: Abrir el estado autónomo de 2.9

```
cd M02/PROYECTO/2.9
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

El estado 2.9 contiene todo lo construido en 2.8. Si reproduces la evolución manualmente, parte del checkpoint anterior y añade únicamente la configuración de este punto.

Paso 2: Separar identidad, acceso e integridad

Antes de escribir índices conviene distinguir tres conceptos:

- una **PK o alternate key** identifica una fila o una clave candidata;
- un **índice** es una estructura de acceso que SQL Server puede usar para localizar datos;
- un **CHECK** impide almacenar valores que violan una regla.

Por eso no se crea en 2.9 otro índice único sobre `NumeroOrden`, `Codigo` o `NumeroCertificado`: esas propiedades ya son alternate keys desde 2.8.

Paso 3: Configurar los cuatro índices de OrdenFabricacion

```
entity.HasIndex(o => o.Cliente)
    .HasDatabaseName("IX_OrdenesFabricacion_Cliente");

entity.HasIndex(o => new { o.Cliente, o.FechaCreacion })
    .HasDatabaseName(
        "IX_OrdenesFabricacion_Cliente_FechaCreacion");

entity.HasIndex(o => o.FechaEntrega)
    .HasFilter("[Estado] = 'Pendiente'")
    .HasDatabaseName(
        "IX_OrdenesFabricacion_FechaEntrega_Pendientes");

entity.HasIndex(o => o.Estado)
    .IncludeProperties(o => new
    {
        o.NumeroOrden,
        o.Cliente,
        o.FechaCreacion
    })
    .HasDatabaseName(
        "IX_OrdenesFabricacion_Estado_Incluye");
```

Cada índice responde a un patrón distinto:

- `Cliente` permite búsquedas directas por cliente.
- `Cliente + FechaCreacion` sirve para filtrar por cliente y ordenar o acotar por fecha.
- `FechaEntrega` filtrado contiene sólo filas cuyo `Estado` es `Pendiente`.
- `Estado` con columnas incluidas puede cubrir consultas que necesitan devolver `NumeroOrden`, `Cliente` y `FechaCreacion` sin convertir esas columnas en parte de la clave del índice.

Paso 4: Comprender el orden del índice compuesto

En:

```
entity.HasIndex(o => new
{
    o.Cliente,
    o.FechaCreacion
});
```

`Cliente` es la primera columna. El índice puede ser útil para consultas que comienzan filtrando por `Cliente` y, después, usan `FechaCreacion`. No debe interpretarse como dos índices independientes.

El índice simple sobre `Cliente` se conserva deliberadamente en el modelo final porque forma parte del diseño del punto; al evaluar redundancia en un sistema real habría que contrastar cargas, planes y mantenimiento antes de eliminar uno.

Paso 5: Sustituir el índice simple de la FK de PlanchaAcero

La relación 1:N había generado un índice simple sobre `OrdenId`. 2.9 configura:

```
entity.HasIndex(x => new
{
    x.OrdenId,
    x.Activa
})
    .HasDatabaseName(
        "IX_PlanchasAcero_OrdenId_Activa");
```

La migración real elimina primero:

```
migrationBuilder.DropIndex(
    name: "IX_PlanchasAcero_OrdenId",
    table: "PlanchasAcero");
```

y crea el compuesto `OrdenId + Activa`. Como `OrdenId` es la primera columna, el compuesto también puede servir a búsquedas que comienzan por esa FK y evita mantener ambos índices en este diseño.

Paso 6: Añadir un índice filtrado sobre Espesor

```
entity.HasIndex(x => x.Espesor)
    .HasFilter("[Activa] = 1")
    .HasDatabaseName(
        "IX_PlanchasAcero_Espesor_Activas");
```

El índice sólo contiene planchas activas. Para que SQL Server pueda aprovecharlo, la consulta debe ser compatible con el predicado del filtro; crear un índice filtrado no obliga al optimizador a usarlo.

Paso 7: Proteger las dimensiones de PlanchaAcero con CHECK

```
entity.ToTable("PlanchasAcero", t =>
{
    t.HasCheckConstraint(
        "CK_PlanchasAcero_Espesor",
        "[Espesor] > 0");

    t.HasCheckConstraint(
        "CK_PlanchasAcero_Ancho",
        "[Ancho] > 0");

    t.HasCheckConstraint(
        "CK_PlanchasAcero_Largo",
        "[Largo] > 0");

    t.HasCheckConstraint(
        "CK_PlanchasAcero_Peso",
        "[Peso] > 0");
});
```

Estas reglas se ejecutan en SQL Server. Aunque una aplicación valide previamente los valores, la base mantiene su propia barrera de integridad.

Paso 8: Configurar índices y CHECK de Aleacion

```
entity.HasIndex(a => a.Nombre)
    .HasDatabaseName("IX_Aleaciones_Nombre");

entity.HasIndex(a => new
{
    a.PorcentajeCarbono,
    a.PorcentajeManganeso
})
    .HasDatabaseName("IX_Aleaciones_Porcentajes");
```

Y sobre la tabla:

```
entity.ToTable("Aleaciones", t =>
{
    t.HasCheckConstraint(
        "CK_Aleaciones_PorcentajeCarbono",
        "[PorcentajeCarbono] >= 0 AND " +
        "[PorcentajeCarbono] <= 2");

    t.HasCheckConstraint(
        "CK_Aleaciones_PorcentajeManganeso",
        "[PorcentajeManganeso] >= 0 AND " +
        "[PorcentajeManganeso] <= 5");
});
```

Código no recibe otro índice: sigue protegido por la alternate key configurada en 2.8.

Paso 9: Mantener las relaciones 1:1 sin duplicar índices manuales

`DetalleOrden.OrdenId` y `CertificadoCalidad.OrdenId` pertenecen a relaciones 1:1 configuradas con `WithOne` y `HasForeignKey`. EF Core ya genera la unicidad necesaria para esas relaciones.

En 2.9 no se vuelve a declarar manualmente otro índice único sobre esas FK. Para `CertificadoCalidad` el cambio nuevo es:

```
entity.HasIndex(c => c.FechaEmision)
    .HasDatabaseName(
        "IX_CertificadosCalidad_FechaEmision");
```

Así se mantiene separada la unicidad estructural de la relación del índice adicional destinado a consultas por fecha.

Paso 10: Configurar los índices de OrdenAleacion

```
entity.HasIndex(x => x.AleacionId)
    .HasDatabaseName(
        "IX_OrdenesAleaciones_AleacionId");

entity.HasIndex(x => x.EstadoRelacion)
    .HasFilter("[EstadoRelacion] = 'Activa'")
    .HasDatabaseName(
        "IX_OrdenesAleaciones_EstadoRelacion_Activas");
```

La PK compuesta empieza por `OrdenFabricacionId`. Por eso no sustituye un índice cuyo primer criterio sea `AleacionId`. El segundo índice reduce el conjunto físico a relaciones cuyo estado es Activa.

Paso 11: Compilar antes de generar la migración

```
dotnet build AceriaData.sln --configuration Release
```

La compilación debe ser correcta antes de comparar el modelo 2.9 con el snapshot de 2.8.

Paso 12: Generar M2_2_9 al reproducir la evolución

Si partes manualmente de 2.8:

```
dotnet ef migrations add M2_2_9
```

El checkpoint final ya contiene:

```
Migrations/20260927204824_M2_2_9.cs
```

No vuelvas a generar esa migración dentro del estado terminado.

Paso 13: Revisar la migración real

M2_2_9 no es una migración genérica. Entre sus operaciones reales aparecen:

```
migrationBuilder.DropIndex(
    name: "IX_PlanchasAcero_OrdenId",
    table: "PlanchasAcero");

migrationBuilder.CreateIndex(
    name: "IX_PlanchasAcero_OrdenId_Activa",
    table: "PlanchasAcero",
    columns: new[] { "OrdenId", "Activa" });

migrationBuilder.CreateIndex(
    name: "IX_OrdenesFabricacion_Estado_Incluye",
    table: "OrdenesFabricacion",
    column: "Estado")
.Annotation(
    "SqlServer:Include",
    new[]
    {
        "NumeroOrden",
        "Cliente",
        "FechaCreacion"
    });

migrationBuilder.CreateIndex(
    name:
        "IX_OrdenesFabricacion_FechaEntrega_Pendientes",
    table: "OrdenesFabricacion",
    column: "FechaEntrega",
    filter: "[Estado] = 'Pendiente'");

migrationBuilder.AddCheckConstraint(
    name: "CK_PlanchasAcero_Espesor",
    table: "PlanchasAcero",
    sql: "[Espesor] > 0");
```

La migración también crea el resto de índices descritos y los CHECK de PlanchaAcero/Aleacion. Su método `Down` revierte esas operaciones y restaura el índice simple `IX_PlanchasAcero_OrdenId`.

Paso 14: Aplicar la migración y comprobar el historial

```
dotnet ef database update
dotnet ef migrations list
```

La lista debe mostrar `M2_2_9` después de `M2_2_8`.

Paso 15: Verificar índices y CHECK en SQL Server

En Visual Studio abre **Ver → Explorador de objetos de SQL Server** y navega a `(localdb)\MSSQLLocalDB → Bases de datos → AceriaDB → Tablas`.

En `OrdenesFabricacion → Índices` comprueba:

- `IX_OrdenesFabricacion_Cliente`;
- `IX_OrdenesFabricacion_Cliente_FechaCreacion`;
- `IX_OrdenesFabricacion_FechaEntrega_Pendientes`;
- `IX_OrdenesFabricacion_Estado_Incluye`.

Repita la inspección en `PlanchasAcero`, `Aleaciones`, `CertificadosCalidad` y `OrdenesAleaciones`. En las propiedades de las tablas verifica también los CHECK generados por `M2_2_9`.

Paso 16: Verificar el filtro y las columnas incluidas

En las propiedades de `IX_OrdenesFabricacion_FechaEntrega_Pendientes` el filtro debe ser:

```
[Estado] = 'Pendiente'
```

En `IX_OrdenesFabricacion_Estado_Incluye` deben figurar como columnas incluidas:

```
NumeroOrden
Cliente
FechaCreacion
```

Las columnas incluidas no cambian la clave lógica del índice; amplían la información que SQL Server puede obtener de él sin volver necesariamente a la tabla base.

Paso 17: Ejecutar e inspeccionar los índices del modelo

```
dotnet run --project AceriaData.Console.csproj --configuration Release
```

El checkpoint consulta el metadata de `OrdenFabricacion`:

```
var entity =
    context.Model.FindEntityType(
        typeof(OrdenFabricacion));

Console.WriteLine(
    $"2.9 OK | Índices: " +
    $"{string.Join(", ", " +
    "entity.GetIndexes()" +
    ".Select(i => i.GetDatabaseName()))}");
```

La salida enumera los índices que EF Core conoce para esa entidad. Esta inspección complementa, pero no sustituye, la verificación del esquema real en SQL Server.

Paso 18: Resolver el reto de Espesor negativo

Reto: demostrar que `CK_PlanchasAcero_Espesor` protege la base incluso si se intenta guardar un valor inválido.

El checkpoint contiene el bloque comentado `RETO 2.9 - ESPESOR NEGATIVO`:

```
context.PlanchasAcero.Add(
    new PlanchaAcero
    {
        OrdenId = orden.Id,
        Espesor = -5.0,
        Ancho = 1000,
        Largo = 2000,
        Peso = 100.000m,
        Activa = true
    });
```

La operación de guardado se prueba de forma controlada:

```
try
{
    context.SaveChanges();

    Console.WriteLine(
        "Reto 2.9 ERROR: SQL Server " +
        "permitió Espesor negativo.");
}
catch (DbUpdateException)
{
    Console.WriteLine(
        "Reto 2.9: Espesor negativo rechazado");
}
```

Resultado esperado:

Reto 2.9: Espesor negativo rechazado

La base ya está preparada mediante Migrations. No se usa `EnsureCreated()` para sustituir el esquema acumulativo.

Paso 19: Observar planes de ejecución sin convertirlos en una promesa

Para estudiar el efecto de los índices puedes ejecutar consultas representativas desde las herramientas de SQL Server y activar el **plan de ejecución real**.

Ejemplos de patrones a observar:

```
SELECT Cliente, FechaCreacion
FROM OrdenesFabricacion
WHERE Cliente = @cliente
ORDER BY FechaCreacion;

SELECT NumeroOrden, Cliente, FechaCreacion
FROM OrdenesFabricacion
WHERE Estado = 'Pendiente';
```

Comprueba qué operadores e índices elige SQL Server. El optimizador decide según estadísticas, cardinalidad, selectividad y coste; la presencia de un índice no garantiza que se utilice.

Paso 20: Delimitar el alcance antes de pasar a 2.10

En 2.9 no se:

- duplican índices sobre las alternate keys de 2.8;
- cambian PK ni relaciones;
- añaden nuevos defaults que ya proceden de 2.7;
- usan índices como sustituto de una regla de identidad;
- fuerza un plan de ejecución concreto;
- usa `EnsureCreated()` para evitar Migrations.

El resultado debe ser exactamente el esquema descrito por `M2_2_9`.

Errores comunes

Error	Causa	Corrección
Duplicar un índice sobre una alternate key	Se confunde identidad con rendimiento	Mantener la alternate key y añadir sólo índices con un propósito distinto

Error	Causa	Corrección
Conservar <code>IX_PlanchasAcero_OrdenId</code> además del compuesto sin necesidad	El compuesto empieza por <code>OrdenId</code>	Seguir la migración real, que elimina el simple
Invertir columnas de un índice compuesto sin analizar consultas	Cambia el prefijo útil del índice	Diseñar el orden según filtros y ordenaciones
Escribir un filtro que no representa las filas buscadas	El índice filtrado deja de ser aplicable	Mantener el predicado coherente con las consultas
Confundir <code>IncludeProperties</code> con columnas de clave	Se interpreta mal el índice cubriente	Diferenciar key columns e included columns
Añadir CHECK sólo en C#	La base acepta datos inválidos desde otros clientes	Mantener la regla también en SQL Server
Suponer que SQL Server siempre usará el índice	El optimizador puede elegir otro plan	Verificar planes con datos y estadísticas reales
Usar <code>EnsureCreated()</code>	Se pierde la evolución del esquema	Aplicar <code>M2_2_9</code> mediante Migrations

Analogía final

Los índices son como distintos catálogos de una acería. Un catálogo por cliente permite localizar rápidamente sus órdenes; otro ordenado por cliente y fecha sirve a búsquedas más específicas; un catálogo filtrado guarda sólo las órdenes pendientes. Las columnas incluidas son información adicional que viaja en el catálogo para evitar consultar otra ficha. Los CHECK son controles de calidad a la entrada: una plancha con espesor o peso imposibles no atraviesa la puerta, venga de la aplicación que venga.

Resultado esperado

Al finalizar 2.9, `AceriaData` conserva las claves de 2.8 y añade los índices definidos por el modelo final: simples, compuestos, filtrados y con columnas incluidas. `M2_2_9` elimina el índice simple redundante de `PlanchaAcero.OrdenId`, crea el compuesto `OrdenId + Activa`, añade el resto de índices y materializa seis CHECK: cuatro para dimensiones/peso de planchas y dos para porcentajes de aleaciones. El reto confirma que SQL Server rechaza un espesor negativo.

Conexión con el siguiente punto

2.10 partirá de este esquema para trabajar con filtros globales de consulta mediante `HasQueryFilter` e `IgnoreQueryFilters`, sin rehacer los índices ni las restricciones ya consolidadas.

Código acumulativo completo del estado 2.9

El siguiente archivo es el `Program.cs` real del estado ejecutable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
```

```

    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";
    [MaxLength(500)]
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("PlanchasAcero")]
public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;
    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }

```

```

    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }
    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
            entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
            entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
            entity.Property(o => o.Observaciones).HasMaxLength(500);
            entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");
            entity.HasIndex(o => o.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");
            entity.HasIndex(o => new { o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");
            entity.HasIndex(o => o.FechaEntrega).HasFilter("[Estado] =

```

```

'Pendiente').HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");
    entity.HasIndex(o => o.Estado).IncludeProperties(o => new { o.NumeroOrden, o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");
});
modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.ToTable("PlanchasAcero", t =>
    {
        t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");
        t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");
        t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");
        t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");
    });
    entity.HasKey(x => x.Id);
    entity.Property(x => x.Peso).HasPrecision(18, 3);
    entity.Property(x => x.Activa).HasDefaultValue(true);
    entity.HasOne(x => x.Orden)
        .WithMany(o => o.Planchas)
        .HasForeignKey(x => x.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
    entity.HasIndex(x => new { x.OrdenId,
x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");
    entity.HasIndex(x => x.Espesor).HasFilter("[Activa] =
1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");
});
modelBuilder.Entity<Aleacion>(entity =>
{
    entity.ToTable("Aleaciones", t =>
    {
        t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND
[PorcentajeCarbono] <= 2");
        t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0
AND [PorcentajeManganeso] <= 5");
    });
    entity.HasKey(a => a.Id);
    entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
    entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
    entity.Property(a => a.Descripcion).HasMaxLength(500);
    entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");
    entity.HasIndex(a => a.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");
    entity.HasIndex(a => new { a.PorcentajeCarbono,
a.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");
});
modelBuilder.Entity<EstadoOrden>(entity =>
{
    entity.ToTable("EstadosOrden");
    entity.HasKey(e => e.Id);
    entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
    entity.Property(e => e.Descripcion).HasMaxLength(250);
    entity.Property(e => e.Activo).HasDefaultValue(true);
});
modelBuilder.Entity<DetalleOrden>(entity =>
{
    entity.ToTable("DetallesOrden");
    entity.HasKey(d => d.Id);
    entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
    entity.Property(d => d.Notas).HasMaxLength(500);
    entity.HasOne(d => d.Orden)
        .WithOne(o => o.Detalle)
        .HasForeignKey<DetalleOrden>(d => d.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});

modelBuilder.Entity<CertificadoCalidad>(entity =>
{
    entity.ToTable("CertificadosCalidad");
    entity.HasKey(c => c.Id);
    entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
    entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
    entity.HasAlternateKey(c =>
c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");

```

```

        entity.HasIndex(c => c.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");
        entity.HasOne(c => c.Orden)
            .WithOne(o => o.Certificado)
            .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
    modelBuilder.Entity<OrdenAleacion>(entity =>
    {
        entity.ToTable("OrdenesAleaciones");
        entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
        entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
        entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
        entity.Property(x =>
x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.OrdenesAleaciones)
            .HasForeignKey(x => x.OrdenFabricacionId)
            .OnDelete(DeleteBehavior.Cascade);
        entity.HasOne(x => x.Aleacion)
            .WithMany(a => a.OrdenesAleaciones)
            .HasForeignKey(x => x.AleacionId)
            .OnDelete(DeleteBehavior.Restrict);
        entity.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");
        entity.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] =
'Activa'").HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");
    });
    }
}

public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOordenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o =>
o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
_context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOordenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOordenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
    }
}

```

```

        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                {
                    sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                    sql.CommandTimeout(60);
                })
                .LogTo(global::System.Console.WriteLine, new[] {
                    "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
        context.Database.EnsureDeleted();
        context.Database.Migrate();

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-M2-0001",
            Cliente = "Constructora del Norte",
            FechaCreacion = DateTime.UtcNow,
            FechaEntrega = DateTime.Today.AddDays(14),
            Estado = "Pendiente",
            Observaciones = "Orden de validación M2",
            Planchas =
            {
                new PlanchaAcero
                {
                    Espesor = 10.5,
                    Ancho = 1500,
                    Largo = 3000,
                    Peso = 371.250m,
                    Activa = true,
                }
            },
            Detalle = new DetalleOrden
            {
                ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
                TemperaturaColada = 1550.5,
                Notas = "Colada principal"
            },
            Certificado = new CertificadoCalidad
            {
                NumeroCertificado = "CERT-0001",
            }
        };
    }
}

```

```

        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden.OrdenesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,
    EstadoRelacion = "Activa"
});
context.OrdenesFabricacion.Add(orden);
context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa",
Activo = true });
context.SaveChanges();

var entity = context.Model.FindEntityType(typeof(OrdenFabricacion))!;
global::System.Console.WriteLine($"2.9 OK | Índices: {string.Join(", ",
entity.GetIndexes().Select(i => i.GetDatabaseName()))}");
}
}

```

Explicación línea a línea del código acumulativo 2.9

Las referencias usan la numeración real de `M02/PROYECTO/2.9/Program.cs`. Se explican todas las líneas no vacías del archivo que compila.

Línea 1: `using System.ComponentModel.DataAnnotations;` → importa el espacio de nombres necesario para este archivo.

Línea 2: `using System.ComponentModel.DataAnnotations.Schema;` → importa el espacio de nombres necesario para este archivo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres necesario para este archivo.

Línea 4: `using Microsoft.Extensions.Configuration;` → importa el espacio de nombres necesario para este archivo.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el espacio de nombres necesario para este archivo.

Línea 6: `using Microsoft.Extensions.Logging;` → importa el espacio de nombres necesario para este archivo.

Línea 8: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres del estado acumulativo.

Línea 10: `[Table("OrdenesFabricacion")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 11: `public class OrdenFabricacion` → declara la clase OrdenFabricacion que forma parte del estado acumulativo.

Línea 12: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 13: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 14: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 15: `[Required]` → marca la propiedad siguiente como requerida.

Línea 16: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 17: `public string NumeroOrden { get; set; } = string.Empty;` → declara la propiedad NumeroOrden de tipo string usada por el modelo o la lógica de negocio.

Línea 18: `[Required]` → marca la propiedad siguiente como requerida.

Línea 19: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 20: `public string Cliente { get; set; } = string.Empty;` → declara la propiedad Cliente de tipo string usada por el modelo o la lógica de negocio.

Línea 21: `public DateTime FechaCreacion { get; set; }` → declara la propiedad FechaCreacion de tipo DateTime usada por el modelo o la lógica de negocio.

Línea 22: `public DateTime? FechaEntrega { get; set; }` → declara la propiedad FechaEntrega de tipo DateTime? usada por el modelo o la lógica de negocio.

Línea 23: `[Required]` → marca la propiedad siguiente como requerida.

Línea 24: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 25: `public string Estado { get; set; } = "Pendiente";` → declara la propiedad Estado de tipo string usada por el modelo o la lógica de negocio.

Línea 26: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 27: `public string? Observaciones { get; set; }` → declara la propiedad Observaciones de tipo string? usada por el modelo o la lógica de negocio.

Línea 28: `public List<PlanchaAcero> Planchas { get; set; } = new();` → declara la propiedad Planchas de tipo List usada por el modelo o la lógica de negocio.

Línea 29: `public DetalleOrden? Detalle { get; set; }` → declara la propiedad Detalle de tipo DetalleOrden? usada por el modelo o la lógica de negocio.

Línea 30: `public CertificadoCalidad? Certificado { get; set; }` → declara la propiedad Certificado de tipo CertificadoCalidad? usada por el modelo o la lógica de negocio.

Línea 31: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo List usada por el modelo o la lógica de negocio.

Línea 32: `}` → cierra el bloque de código actual.

Línea 34: `[Table("PlanchasAcero")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 35: `public class PlanchaAcero` → declara la clase PlanchaAcero que forma parte del estado acumulativo.

Línea 36: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 37: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 38: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 39: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int usada por el modelo o la lógica de negocio.

Línea 40: `public double Espesor { get; set; }` → declara la propiedad Espesor de tipo double usada por el modelo o la lógica de negocio.

Línea 41: `public double Ancho { get; set; }` → declara la propiedad Ancho de tipo double usada por el modelo o la lógica de negocio.

Línea 42: `public double Largo { get; set; }` → declara la propiedad Largo de tipo double usada por el modelo o la lógica de negocio.

Línea 43: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 44: `public decimal Peso { get; set; }` → declara la propiedad Peso de tipo decimal usada por el modelo o la lógica de negocio.

Línea 45: `public bool Activa { get; set; } = true;` → declara la propiedad Activa de tipo bool usada por el modelo o la lógica de negocio.

Línea 46: `[ForeignKey(nameof(OrdenId))]` → vincula la navegación siguiente con su clave foránea.

Línea 47: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion usada por el modelo o la lógica de negocio.

Línea 48: `}` → cierra el bloque de código actual.

Línea 50: `[Table("Aleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 51: `public class Aleacion` → declara la clase Aleacion que forma parte del estado acumulativo.

Línea 52: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 53: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 54: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 55: `[Required]` → marca la propiedad siguiente como requerida.

Línea 56: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 57: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string usada por el modelo o la lógica de negocio.

Línea 58: `[Required]` → marca la propiedad siguiente como requerida.

Línea 59: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 60: `public stringCodigo { get; set; } = string.Empty;` → declara la propiedad Codigo de tipo string usada por el modelo o la lógica de negocio.

Línea 61: `public double PorcentajeCarbono { get; set; }` → declara la propiedad PorcentajeCarbono de tipo double usada por el modelo o la lógica de negocio.

Línea 62: `public double PorcentajeManganeso { get; set; }` → declara la propiedad PorcentajeManganeso de tipo double usada por el modelo o la lógica de negocio.

Línea 63: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 64: `public string? Descripcion { get; set; }` → declara la propiedad Descripcion de tipo string? usada por el modelo o la lógica de negocio.

Línea 65: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo List usada por el modelo o la lógica de negocio.

Línea 66: `}` → cierra el bloque de código actual.

Línea 68: `[Table("EstadosOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 69: `public class EstadoOrden` → declara la clase EstadoOrden que forma parte del estado acumulativo.

Línea 70: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 71: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 72: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 73: `[Required]` → marca la propiedad siguiente como requerida.

Línea 74: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 75: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string usada por el modelo o la lógica de negocio.

Línea 76: `[MaxLength(250)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 77: `public string Descripcion { get; set; } = string.Empty;` → declara la propiedad Descripcion de tipo string usada por el modelo o la lógica de negocio.

Línea 78: `public bool Activo { get; set; } = true;` → declara la propiedad Activo de tipo bool usada por el modelo o la lógica de negocio.

Línea 79: `}` → cierra el bloque de código actual.

Línea 81: `[Table("DetallesOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 82: `public class DetalleOrden` → declara la clase DetalleOrden que forma parte del estado acumulativo.

Línea 83: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 84: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 85: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 86: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int usada por el modelo o la lógica de negocio.

Línea 87: `[Required]` → marca la propiedad siguiente como requerida.

Línea 88: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 89: `public string ComposicionQuimica { get; set; } = string.Empty;` → declara la propiedad ComposicionQuimica de tipo string usada por el modelo o la lógica de negocio.

Línea 90: `public double TemperaturaColada { get; set; }` → declara la propiedad TemperaturaColada de tipo double usada por el modelo o la lógica de negocio.

Línea 91: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 92: `public string? Notas { get; set; }` → declara la propiedad Notas de tipo string? usada por el modelo o la lógica de negocio.

Línea 93: `[ForeignKey(nameof(OrdenId))]` → vincula la navegación siguiente con su clave foránea.

Línea 94: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion usada por el modelo o la lógica de negocio.

Línea 95: `}` → cierra el bloque de código actual.

Línea 97: `[Table("CertificadosCalidad")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 98: `public class CertificadoCalidad` → declara la clase CertificadoCalidad que forma parte del estado acumulativo.

Línea 99: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 100: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 101: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 102: `public int OrdenId { get; set; }` → declara la propiedad `OrdenId` de tipo `int` usada por el modelo o la lógica de negocio.

Línea 103: `[Required]` → marca la propiedad siguiente como requerida.

Línea 104: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 105: `public string NumeroCertificado { get; set; } = string.Empty;` → declara la propiedad `NumeroCertificado` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 106: `public DateTime FechaEmision { get; set; }` → declara la propiedad `FechaEmision` de tipo `DateTime` usada por el modelo o la lógica de negocio.

Línea 107: `[Required]` → marca la propiedad siguiente como requerida.

Línea 108: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 109: `public string OrganismoCertificador { get; set; } = string.Empty;` → declara la propiedad `OrganismoCertificador` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 110: `[ForeignKey(nameof(OrdenId))]` → vincula la navegación siguiente con su clave foránea.

Línea 111: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` usada por el modelo o la lógica de negocio.

Línea 112: `}` → cierra el bloque de código actual.

Línea 114: `[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]` → declara una clave primaria compuesta.

Línea 115: `[Table("OrdenesAleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 116: `public class OrdenAleacion` → declara la clase `OrdenAleacion` que forma parte del estado acumulativo.

Línea 117: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 118: `public int OrdenFabricacionId { get; set; }` → declara la propiedad `OrdenFabricacionId` de tipo `int` usada por el modelo o la lógica de negocio.

Línea 119: `public int AleacionId { get; set; }` → declara la propiedad `AleacionId` de tipo `int` usada por el modelo o la lógica de negocio.

Línea 120: `public DateTime FechaAsignacion { get; set; } = DateTime.Now;` → declara la propiedad `FechaAsignacion` de tipo `DateTime` usada por el modelo o la lógica de negocio.

Línea 121: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 122: `public decimal CantidadUtilizada { get; set; }` → declara la propiedad `CantidadUtilizada` de tipo `decimal` usada por el modelo o la lógica de negocio.

Línea 123: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 124: `public string EstadoRelacion { get; set; } = "Activa";` → declara la propiedad `EstadoRelacion` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 125: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` usada por el modelo o la lógica de negocio.

Línea 126: `public Aleacion Aleacion { get; set; } = null!;` → declara la propiedad `Aleacion` de tipo `Aleacion` usada por el modelo o la lógica de negocio.

Línea 127: `}` → cierra el bloque de código actual.

Línea 129: `public class AceriaDbContext : DbContext` → declara la clase `AceriaDbContext` que forma parte del estado acumulativo.

Línea 130: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 131:

`public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → recibe las opciones del DbContext y las pasa a la clase base.

Línea 133: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 134: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 135: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 136: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 137: `public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 138: `public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 139: `public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 142: `protected override void OnModelCreating(ModelBuilder modelBuilder)` → entra en la configuración explícita del modelo.

Línea 143: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 144: `base.OnModelCreating(modelBuilder);` → entra en la configuración explícita del modelo.

Línea 145: `modelBuilder.Entity<OrdenFabricacion>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 146: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 147: `entity.ToTable("OrdenesFabricacion");` → asigna explícitamente la tabla usada por la entidad.

Línea 148: `entity.HasKey(o => o.Id);` → define la clave primaria.

Línea 149: `entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 150: `entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 151: `entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 152: `entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");` → selecciona una propiedad escalar para continuar su configuración.

Línea 153: `entity.Property(o => o.Observaciones).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 154: `entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");` → define una clave alternativa única.

Línea 155: `entity.HasIndex(o => o.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");` → define un índice sobre las propiedades indicadas.

Línea 156: `entity.HasIndex(o => new { o.Cliente, o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");` → define un índice sobre las propiedades indicadas.

Línea 157: `entity.HasIndex(o => o.FechaEntrega).HasFilter("[Estado] = 'Pendiente']").HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");` → define un índice sobre las propiedades indicadas.

Línea 158: `entity.HasIndex(o => o.Estado).IncludeProperties(o => new { o.NumeroOrden, o.Cliente, o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");` → define un índice sobre las propiedades indicadas.

Línea 159: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 160: `modelBuilder.Entity<PlanchaAcero>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 161: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 162: `entity.ToTable("PlanchasAcero", t =>` → asigna explícitamente la tabla usada por la entidad.

Línea 163: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 164: `t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");` → añade una restricción CHECK en SQL Server.

Línea 165: `t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");` → añade una restricción CHECK en SQL Server.

Línea 166: `t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");` → añade una restricción CHECK en SQL Server.

Línea 167: `t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");` → añade una restricción CHECK en SQL Server.

Línea 168: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 169: `entity.HasKey(x => x.Id);` → define la clave primaria.

Línea 170: `entity.Property(x => x.Peso).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 171: `entity.Property(x => x.Activa).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 172: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 173: `.WithMany(o => o.Planchas)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 174: `.HasForeignKey(x => x.OrdenId)` → declara explícitamente la clave foránea.

Línea 175: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 176: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 177: `entity.HasIndex(x => new { x.OrdenId, x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");` → define un índice sobre las propiedades indicadas.

Línea 178: `entity.HasIndex(x => x.Espesor).HasFilter("[Activa] = 1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");` → define un índice sobre las propiedades indicadas.

Línea 179: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 180: `modelBuilder.Entity<Aleacion>(entity => → selecciona una entidad para configurarla con Fluent API.`

Línea 181: `{ → abre el bloque de código asociado a la declaración o instrucción anterior.`

Línea 182: `entity.ToTable("Aleaciones", t => → asigna explícitamente la tabla usada por la entidad.`

Línea 183: `{ → abre el bloque de código asociado a la declaración o instrucción anterior.`

Línea 184: `t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND [PorcentajeCarbono] <= 2"); → añade una restricción CHECK en SQL Server.`

Línea 185: `t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0 AND [PorcentajeManganeso] <= 5"); → añade una restricción CHECK en SQL Server.`

Línea 186: `}); → cierra la lambda o configuración encadenada y finaliza la llamada.`

Línea 187: `entity.HasKey(a => a.Id); → define la clave primaria.`

Línea 188: `entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100); → selecciona una propiedad escalar para continuar su configuración.`

Línea 189: `entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20); → selecciona una propiedad escalar para continuar su configuración.`

Línea 190: `entity.Property(a => a.Descripcion).HasMaxLength(500); → selecciona una propiedad escalar para continuar su configuración.`

Línea 191: `entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo"); → define una clave alternativa única.`

Línea 192: `entity.HasIndex(a => a.Nombre).HasDatabaseName("IX_Aleaciones_Nombre"); → define un índice sobre las propiedades indicadas.`

Línea 193: `entity.HasIndex(a => new { a.PorcentajeCarbono, a.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes"); → define un índice sobre las propiedades indicadas.`

Línea 194: `}); → cierra la lambda o configuración encadenada y finaliza la llamada.`

Línea 195: `modelBuilder.Entity<EstadoOrden>(entity => → selecciona una entidad para configurarla con Fluent API.`

Línea 196: `{ → abre el bloque de código asociado a la declaración o instrucción anterior.`

Línea 197: `entity.ToTable("EstadosOrden"); → asigna explícitamente la tabla usada por la entidad.`

Línea 198: `entity.HasKey(e => e.Id); → define la clave primaria.`

Línea 199: `entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50); → selecciona una propiedad escalar para continuar su configuración.`

Línea 200: `entity.Property(e => e.Descripcion).HasMaxLength(250); → selecciona una propiedad escalar para continuar su configuración.`

Línea 201: `entity.Property(e => e.Activo).HasDefaultValue(true); → selecciona una propiedad escalar para continuar su configuración.`

Línea 202: `}); → cierra la lambda o configuración encadenada y finaliza la llamada.`

Línea 203: `modelBuilder.Entity<DetalleOrden>(entity => → selecciona una entidad para configurarla con Fluent API.`

Línea 204: `{ → abre el bloque de código asociado a la declaración o instrucción anterior.`

Línea 205: `entity.ToTable("DetallesOrden"); → asigna explícitamente la tabla usada por la entidad.`

Línea 206: `entity.HasKey(d => d.Id); → define la clave primaria.`

Línea 207: `entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 208: `entity.Property(d => d.Notas).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 209: `entity.HasOne(d => d.Orden)` → inicia la configuración de una navegación de referencia.

Línea 210: `.WithOne(o => o.Detalle)` → establece el extremo de referencia de una relación uno-a-uno.

Línea 211: `.HasForeignKey<DetalleOrden>(d => d.OrdenId)` → declara explícitamente la clave foránea.

Línea 212: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 213: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 214: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 216: `modelBuilder.Entity<CertificadoCalidad>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 217: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 218: `entity.ToTable("CertificadosCalidad");` → asigna explícitamente la tabla usada por la entidad.

Línea 219: `entity.HasKey(c => c.Id);` → define la clave primaria.

Línea 220: `entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 221: `entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 222: `entity.HasAlternateKey(c => c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");` → define una clave alternativa única.

Línea 223: `entity.HasIndex(c => c.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");` → define un índice sobre las propiedades indicadas.

Línea 224: `entity.HasOne(c => c.Orden)` → inicia la configuración de una navegación de referencia.

Línea 225: `.WithOne(o => o.Certificado)` → establece el extremo de referencia de una relación uno-a-uno.

Línea 226: `.HasForeignKey<CertificadoCalidad>(c => c.OrdenId)` → declara explícitamente la clave foránea.

Línea 227: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 228: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 229: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 230: `modelBuilder.Entity<OrdenAleacion>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 231: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 232: `entity.ToTable("OrdenesAleaciones");` → asigna explícitamente la tabla usada por la entidad.

Línea 233: `entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });` → define la clave primaria.

Línea 234: `entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 235: `entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 236: `entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");` → selecciona una propiedad escalar para continuar su configuración.

Línea 237: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 238: `.WithMany(o => o.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 239: `.HasForeignKey(x => x.OrdenFabricacionId)` → declara explícitamente la clave foránea.

Línea 240: `.OnDelete(DeleteBehavior.Cascade);` → define el comportamiento de eliminación de la relación.

Línea 241: `entity.HasOne(x => x.Aleacion)` → inicia la configuración de una navegación de referencia.

Línea 242: `.WithMany(a => a.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 243: `.HasForeignKey(x => x.AleacionId)` → declara explícitamente la clave foránea.

Línea 244: `.OnDelete(DeleteBehavior.Restrict);` → define el comportamiento de eliminación de la relación.

Línea 245: `entity.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");` → define un índice sobre las propiedades indicadas.

Línea 246: `entity.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] = 'Activa'").HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");` → define un índice sobre las propiedades indicadas.

Línea 247: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 248: `}` → cierra el bloque de código actual.

Línea 249: `}` → cierra el bloque de código actual.

Línea 251: `public interface IOrdenRepositorio` → declara el contrato IOrdenRepositorio usado por la aplicación.

Línea 252: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 253: `List<OrdenFabricacion> ObtenerTodas();` → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 254: `OrdenFabricacion? ObtenerPorId(int id);` → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.

Línea 255: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara en el contrato la búsqueda por el identificador de negocio NumeroOrden.

Línea 256: `void Agregar(OrdenFabricacion orden);` → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.

Línea 257: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.

Línea 258: `int Guardar();` → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.

Línea 259: `}` → cierra el bloque de código actual.

Línea 261: `public sealed class OrdenRepositorio : IOrdenRepositorio` → declara la clase `OrdenRepositorio` que forma parte del estado acumulativo.

Línea 262: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 263: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext`.

Línea 264: `public OrdenRepositorio(AceriaDbContext context) => _context = context;` → inyecta `AceriaDbContext` en el repositorio y lo conserva como dependencia de persistencia.

Línea 266: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → ordena el resultado antes de materializarlo.

Línea 267: `public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave primaria mediante `Find`, aprovechando primero el `ChangeTracker` y después la base de datos.

Línea 268: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → recupera la primera coincidencia o `null`.

Línea 269: `public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);` → registra una entidad nueva para insertarla al guardar.

Línea 270: `public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);` → marca una entidad para borrado al guardar.

Línea 271: `public int Guardar() => _context.SaveChanges();` → confirma en SQL Server los cambios del `ChangeTracker`.

Línea 272: `}` → cierra el bloque de código actual.

Línea 274: `public interface IServicioOrdenes` → declara el contrato `IServicioOrdenes` usado por la aplicación.

Línea 275: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 276: `string ObtenerResumen(int ordenId);` → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 277: `}` → cierra el bloque de código actual.

Línea 279: `public sealed class ServicioOrdenes : IServicioOrdenes` → declara la clase `ServicioOrdenes` que forma parte del estado acumulativo.

Línea 280: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 281: `private readonly IOrdenRepositorio _repositorio;` → declara el campo privado `_repositorio` de tipo `IOrdenRepositorio`.

Línea 282: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext`.

Línea 283: `public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)` → declara el constructor que recibe por inyección el repositorio de órdenes y el `DbContext`.

Línea 284: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 285: `_repositorio = repositorio;` → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.

Línea 286: `_context = context;` → guarda el `DbContext` inyectado para consultar los datos necesarios durante el servicio.

Línea 287: `}` → cierra el bloque de código actual.

Línea 289: `public string ObtenerResumen(int ordenId)` → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.

Línea 290: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 291: `var orden = _repositorio.ObtenerPorId(ordenId);` → declara una variable local y almacena el resultado de la expresión.

Línea 292: `if (orden is null) return $"Orden {ordenId} no encontrada";` → evalúa una condición antes de ejecutar el bloque asociado.

Línea 293: `var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);` → cuenta las filas que cumplen la consulta.

Línea 294: `return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";` → devuelve el resultado al código llamador.

Línea 295: `}` → cierra el bloque de código actual.

Línea 296: `}` → cierra el bloque de código actual.

Línea 298: `public static class Program` → declara la clase Program que forma parte del estado acumulativo.

Línea 299: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 300: `public static void Main()` → declara el punto de entrada ejecutable de la aplicación de consola.

Línea 301: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 302: `var configuration = new ConfigurationBuilder()` → inicia la construcción de la configuración de la aplicación.

Línea 303: `.SetBasePath(AppContext.BaseDirectory)` → establece la ruta base para los archivos de configuración.

Línea 304: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)` → incorpora appsettings.json.

Línea 305: `.AddEnvironmentVariables()` → incorpora variables de entorno.

Línea 306: `.Build();` → materializa el objeto construido por la cadena anterior.

Línea 308: `var connectionString = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena AceriaDB desde configuración.

Línea 309: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.

Línea 311: `var services = new ServiceCollection();` → crea la colección de servicios.

Línea 312: `services.AddDbContext<AceriaDbContext>(options =>` → registra el DbContext en inyección de dependencias.

Línea 313: `options` → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de AddDbContext.

Línea 314: `.UseSqlServer(connectionString, sql =>` → selecciona SQL Server como proveedor de EF Core.

Línea 315: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 316: `sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);` → habilita reintentos ante fallos transitorios.

Línea 317: `sql.CommandTimeout(60);` → establece el timeout de los comandos SQL.

Línea 318: `})` → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.

Línea 319: `.LogTo(global::System.Console.WriteLine, new[]
{ "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)` → configura el logging de EF Core.

Línea 320: `.EnableDetailedErrors();` → activa errores detallados.

Línea 322: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();` → registra un servicio con ciclo de vida scoped.

Línea 323: `services.AddScoped<IServicioOrdenes, ServicioOrdenes>();` → registra un servicio con ciclo de vida scoped.

Línea 325: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → construye el ServiceProvider con validación de servicios y garantiza su liberación al salir del ámbito mediante using var.

Línea 326: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 327: `ValidateScopes = true,` → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.

Línea 328: `ValidateOnBuild = true` → obliga al contenedor a validar las resoluciones registradas al construir el ServiceProvider.

Línea 329: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 331: `using var scope = provider.CreateScope();` → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.

Línea 332: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → resuelve un servicio obligatorio del contenedor.

Línea 333: `context.Database.EnsureDeleted();` → reinicia la base del laboratorio para una ejecución reproducible.

Línea 334: `context.Database.Migrate();` → aplica el historial de migraciones pendiente.

Línea 336: `var orden = new OrdenFabricacion` → declara una variable local y almacena el resultado de la expresión.

Línea 337: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 338: `NumeroOrden = "0F-M2-0001",` → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.

Línea 339: `Cliente = "Constructora del Norte",` → asigna el cliente de la orden utilizada en el escenario práctico del punto.

Línea 340: `FechaCreacion = DateTime.UtcNow,` → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.

Línea 341: `FechaEntrega = DateTime.Today.AddDays(14),` → establece una fecha de entrega prevista catorce días después de la fecha local actual.

Línea 342: `Estado = "Pendiente",` → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.

Línea 343: `Observaciones = "Orden de validación M2",` → añade texto opcional para comprobar el mapeo de Observaciones.

Línea 344: `Planchas =` → inicia el inicializador de la colección de planchas asociadas a la orden.

Línea 345: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 346: `new PlanchaAcero` → crea una nueva plancha dentro del inicializador de la colección Planchas de la orden.

Línea 347: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 348: `Espesor = 10.5,` → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.

Línea 349: `Ancho = 1500,` → asigna el ancho de la plancha usada en el escenario de persistencia.

Línea 350: `Largo = 3000,` → asigna el largo de la plancha usada en el escenario de persistencia.

Línea 351: `Peso = 371.250m,` → asigna un peso decimal con tres posiciones para validar la precisión configurada.

Línea 352: `Activa = true,` → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.

Línea 353: `}` → cierra el bloque de código actual.

Línea 354: `},` → continúa el inicializador o la llamada del bloque actual.

Línea 355: `Detalle = new DetalleOrden` → crea el detalle uno-a-uno que se persistirá junto con la orden principal.

Línea 356: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 357: `ComposicionQuimica = "C: 0.45%, Mn: 0.75%",` → asigna la composición química utilizada para verificar la entidad DetalleOrden.

Línea 358: `TemperaturaColada = 1550.5,` → asigna la temperatura de colada del detalle de fabricación.

Línea 359: `Notas = "Colada principal"` → asigna las notas opcionales del detalle asociado a la orden.

Línea 360: `},` → continúa el inicializador o la llamada del bloque actual.

Línea 361: `Certificado = new CertificadoCalidad` → crea el certificado uno-a-uno asociado a la orden del escenario.

Línea 362: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 363: `NumeroCertificado = "CERT-0001",` → asigna el identificador natural del certificado de calidad.

Línea 364: `FechaEmision = DateTime.Today,` → asigna la fecha de emisión del certificado usado en la prueba.

Línea 365: `OrganismoCertificador = "Laboratorio Aceria"` → asigna el organismo que emite el certificado de calidad.

Línea 366: `},` → continúa el inicializador o la llamada del bloque actual.

Línea 367: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 369: `var aleacion = new Aleacion` → declara una variable local y almacena el resultado de la expresión.

Línea 370: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 371: `Nombre = "AISI 1045",` → asigna el nombre comercial de la aleación utilizada en el escenario.

Línea 372: `Codigo = "A1045",` → asigna el código natural único de la aleación.

Línea 373: `PorcentajeCarbono = 0.45,` → asigna el porcentaje de carbono dentro del rango permitido por la restricción del modelo.

Línea 374: `PorcentajeManganeso = 0.75,` → asigna el porcentaje de manganeso dentro del rango permitido por la restricción del modelo.

Línea 375: `Descripcion = "Acero medio en carbono"` → asigna una descripción opcional a la aleación.

Línea 376: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 377: `orden.OrdenesAleaciones.Add(new OrdenAleacion` → registra una entidad nueva para insertarla al guardar.

Línea 378: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 379: `Aleacion = aleacion,` → conecta la entidad intermedia `OrdenAleacion` con la aleación creada para el escenario.

Línea 380: `CantidadUtilizada = 1500.500m,` → registra la cantidad utilizada con tres decimales, coherente con `HasPrecision(18, 3)`.

Línea 381: `EstadoRelacion = "Activa"` → marca como activa la relación entre la orden y la aleación.

Línea 382: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 383: `context.OrdenesFabricacion.Add(orden);` → registra una entidad nueva para insertarla al guardar.

Línea 384: `context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });` → registra una entidad nueva para insertarla al guardar.

Línea 385: `context.SaveChanges();` → confirma en SQL Server los cambios del `ChangeTracker`.

Línea 387: `var entity = context.Model.FindEntityType(typeof(OrdenFabricacion))!;` → declara una variable local y almacena el resultado de la expresión.

Línea 388: `global::System.Console.WriteLine($"2.9 OK | Índices: {string.Join(", ", entity.GetIndexes().Select(i => i.GetDatabaseName()))}");` → escribe la evidencia de ejecución usada para validar este estado.

Línea 389: `}` → cierra el bloque de código actual.

Línea 390: `}` → cierra el bloque de código actual.